

Contents

1 week 4: clustering & decision trees	1
1.1 k-means	1
1.1.1 introduction to k-means algorithm	1
1.1.2 plotting our data in python	2
1.1.3 intuition on the "centroid"	2
1.1.4 introduce 'K'	3
1.1.5 do it in Python	3
1.1.6 scaling & comparing	4
1.1.7 in groups: deliberately change k to find other possibilities	5
1.1.8 question: does k-means discover groups, or does it construct groups?	6
1.2 break	6
1.3 decision tree intuition	6
1.4 install git	7
1.5 white collar crime	7
1.5.1 setting up a repo from an ML project	7
1.5.2 investigating an ML project	8
1.5.3 moving to predict-notebook.ipynb	9
1.6 next week: evaluation	10

1 week 4: clustering & decision trees

1.1 k-means

1.1.1 introduction to k-means algorithm

k-means is a new algorithm, different from the regression/classification work you've already been doing:

- > **Linear regression:** predict a number, supervised.
 - predicts the slope of a line, linear relation between $x \rightarrow y$
- > **Logistic regression:** predict a category/class, supervised.
 - predicts the decision boundary between two types, y is true/false.
- > **K-means:** predict clusters, unsupervised.
 - predicts where to draw y lines to create groupings of data x .

These different algorithms work on different spreads of data, and different question of prediction.

A note on unsupervised learning:

- **K-means doesn't know what the groups mean. It only knows which observations are similar to one another.**
- the difference between **a computationally produced grouping and an interpretive category.**

1.1.2 plotting our data in python

Suppose we have people described by:

- 'age'
- 'annual_income'

```
import pandas as pd
import matplotlib.pyplot as plt

df = pd.read_csv("people.csv")

plt.scatter(df["age"], df["annual_income"])
plt.xlabel("Age")
plt.ylabel("Annual income")
plt.show()
```

Identify possible clusters

- where are there potential groupings in this data?

1.1.3 intuition on the "centroid"

Imagine three clusters of dots.

K-means essentially does this:

1. Pick 'k' centers. (random)
2. Assign each data point to the nearest center.
3. Move each center to the middle of its assigned data points.
4. Repeat.

5. Stop when the centers aren't changing much.

You can describe the algorithm without equations:

- initialize centers randomly
- assign points to nearest center
- move centers to be in the middle of the assigned points
- re-assign points
- re-move centers

The iterative nature is much easier to understand visually than mathematically.

1.1.4 introduce 'K'

How many groups should we look for?

- that's where 'k' comes in.
- K is the number of clusters we are asking the algorithm to find.

At first, $k = 3$.

- However, K-means doesn't automatically know that there are three meaningful groups. We tell it to look for three.

1.1.5 do it in Python

```
from sklearn.cluster import KMeans

X = df[["age", "annual_income"]]

model = KMeans(n_clusters=3, random_state=42)

model.fit(X)
```

Then create a new column for cluster labels

```
df["cluster"] = model.labels_
df
```

What does cluster 1 mean?

- We don't know yet
- The algorithm has created the grouping
- But we have to interpret it.

Let's make the clustering visible.

```
plt.scatter( df["age"], df["annual_income"], c=df["cluster"])

plt.xlabel("Age")
plt.ylabel("Annual income")
plt.show()
```

Now that you can see the groups, does this look like a reasonable grouping?

What does each cluster represent? Calculate group characteristics.

```
df.groupby("cluster")[["age", "annual_income"]].mean()
```

Now start to interpret:

- Cluster 0 = older/lower income
- Cluster 1 = younger/lower income
- Cluster 2 = middle-aged/higher income

Those labels are our interpretation, not something k-means discovered.

1.1.6 scaling & comparing

It is important to scale values, like we did with logistic regression

- the high numbers of the salary are affecting the creation of the clusters
 - the numbers we give an algorithm affect the patterns it sees.
 - we need to scale the larger values, like income
- K-means uses **distance**, so income can dominate the calculation.

Introduce:

```

from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()

X_scaled = scaler.fit_transform(X)

X_scaled # compare to just X

    Then, re-train the model.

model = KMeans(n_clusters=3, random_state=42)

model.fit(X_scaled)

df["cluster"] = model.labels_  model = KMeans(n_clusters=3, random_state=42)

model.fit(X_scaled)

df["cluster"] = model.labels_

    Now, compare the plots.

plt.scatter( df["age"], df["annual_income"], c=df["cluster"])

plt.xlabel("Age")
plt.ylabel("Annual income")
plt.show()

```

1.1.7 in groups: deliberately change k to find other possibilities

```

model = KMeans(n_clusters=2, random_state=42)
model.fit(X)
df["cluster"] = model.labels_
df
plt.scatter( df["age"], df["annual_income"], c=df["cluster"])

plt.xlabel("Age")
plt.ylabel("Annual income")
plt.show()

```

Try different values for K and compare the plots. Which one is best?

This is a great moment to complicate the idea of "machine learning finds patterns." Because **the answer depends partly on decisions made by the researcher.**

1.1.8 question: does k-means discover groups, or does it construct groups?

K-means:

- doesn't know what a group means
- doesn't know whether the grouping is socially meaningful
- doesn't know whether the features are appropriate
- doesn't know whether 'k = 3' is conceptually justified
- doesn't know whether the data itself reflects bias
- simply optimizes a mathematical criterion based on the representations we provide

So the computational output isn't the end of the analysis. The interpretation comes afterward.

1.2 break

1.3 decision tree intuition

- create a graph of when you can go skiing in different parts of the world.
 - latitude on the Y axis; month of the year on the X axis
- create splits of the data, and write conditions for each split
 - first, create condition of $y > 30$
 - * this is the first split of the decision tree.
 - then, in that subgroup, split off $x < 3$.
 - * this is the second split in the decision tree.
 - so, if $y > 30$ & $x < 3$, then yes you can ski.
 - you begin to partition out the dataset this way.

- * by asking questions about each quantrant, you fill out the decision tree.

- what is a random forrest?
 - a collection of decision trees.
 - decision trees tend to overfit, which means they predict only what they have seen in their training data, they don't generalize.

1.4 install git

- check if you have it, `git --version`
- if not, install via homebrew

1.5 white collar crime

- you're going to learn how to download and run an ML project from github on your own computer
- you're also going to learn how to understand what's going on, and to change some of the variables

1.5.1 setting up a repo from an ML project

- First, clone the repo on github
 - `https://github.com/gofilipa/whitecollarcrime`
- cd into your Desktop (or wherever you save projects).
- run:
 - `git clone https://github.com/gofilipa/whitecollarcrime.git`
- then, cd into the repo and set up your enviornment:

```
# create * activate an env
conda create --name whitecollar
conda activate whitecollar
```

```
# install packages from the env
cd whitecollar
pip install -r requirements.txt
```

```
# if you get a permissions error:
sudo python -m pip install -r requirements.txt
```

1.5.2 investigating an ML project

1. what are **three main components** to an ML experiment/project?

- training data, made of predictors and target
- model
- evaluation metric

```
X = predictors
y = target
model = machine_learning_algorithm
score = evaluation metric
```

2. from the reading, in partners, 10 minutes **For this particular experiment, what is X, y, model, and score?**

```
X → crime statistics, liquor licenses,
    tax-exempt organizations,
    investment advisers
    (also fines)
```

```
y → financial crime: yes/no
    (also: fines & crimes)
```

```
model → Random Forest
        (also linear regression?)
```

```
score → Cross Validation
```

3. investigate the script, in partners, 20m We are going to focus on the first **RandomForest** algorithm, which predicts the likelihood of crime.

Now, examine the script, and see how these are defined. What goes into them?

- For each variable, explain what it variable is, and how it is used in the script (what it passed into it, and where its outputs go).

- Examine any other variables, but focus on these highlighted below:

```
X
y
m
# examine functions:
fit()
predict()
score()
```

Variable	What it is	How it is used
'X'	Input features/predictors	training data given to model
'y'	Target	Whether crime occurred ('True'/'False')
'features'	List of predictor names	variables that become 'X'
'y_fines'	Target for fine amount	Used for the second prediction model
'y_allegations'	Target for allegation types	Used for the third prediction model
'X_pred'	Features for the geographic areas	Given to models to generate predictions
'CRIME_TYPES'	List of crime categories	Defines some features that go into 'X'
'data'	Dict w/ information for geog area	Stores raw/processed data before model
'predictions'	Dictionary of model predictions	Stores predictions for each geog area
'types'	Names of allegation categories	the allegation predictions

What is:

- cross validation
- RandomForestClassifier

There is a problem with the evaluation metric in this script. What is it?

1.5.3 moving to predict-notebook.ipynb

- load up all of the imports and data processing in individual cells
- when arriving to the first model, stop adding code
- import new models for Logistic Regression and Decision Tree
- change to different models and compare

- try logistic regression and decision trees
- what does this tell you?
- next step: examine what exactly is happening to the data in the data processing stage.

1.6 next week: evaluation